Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Carcassonne – Python

Rapport préliminaire

Anthony BEAUVIEUX, Tony HUANG, Maxence SALIN, Clément SIMON, Loïc THIN

1 Présentation du jeu

1.1 Contexte

Carcassonne est un jeu de stratégie et de placement de tuiles créé par Klaus-Jürgen Wrede en 2000, conçu pour 2 à 6 joueurs. Il s'agit d'un jeu stochastique à information parfaite, c'est-à-dire un jeu où le hasard intervient dans le tirage des tuiles mais où l'ensemble des éléments présents sur le plateau est visible de tous les joueurs.



FIGURE 1 – Tuile de départ

La version considérée ici utilise les 72 tuiles du jeu de base. Parmi ces tuiles, se trouve la tuile de départ (figure 1) qui est placée au centre du plateau avant le début de la partie. Cette tuile est la même pour chaque partie et comporte une *City* en haut (couleur marron), une *Road* qui traverse la tuile de gauche à droite et un *Field* en bas. Chaque joueur dispose de 7 pions de sa couleur pour marquer des points au cours de la partie. Le but du jeu est que les joueurs construisent progressivement le plateau, à tour de rôle, en posant des tuiles de manière cohérente et en plaçant stratégiquement leurs pions pour accumuler des points. Le vainqueur est le joueur qui totalise le plus de points à la fin de la partie.

1.2 Historique et état de l'art

Carcassonne a connu un succès commercial important depuis sa création, avec de nombreuses extensions et variantes publiées au fil des années. Il a également suscité un intérêt académique significatif, notamment dans les domaines de l'intelligence artificielle et de la théorie des jeux, en raison de sa complexité stratégique et de sa nature à la fois déterministe et stochastique. Ainsi, plusieurs travaux académiques et implémentations logicielles ont étudié les problématiques liées à l'intelligence artificielle pour Carcassonne. L'article de Heyden et al.[6] propose une analyse détaillée de l'arbre de jeu et des techniques de recherche heuristique adaptées aux contraintes de ce jeu, tandis que d'autres contributions explorent des approches Monte Carlo ou Expectimax pour traiter l'incertitude issue de la pioche.

1.3 Règles

Les règles de **Carcassonne** utilisées sont celles de la version 2012[3].

1.3.1 Spécificités

Tuile

Chaque tuile possède quatre côtés ainsi qu'un centre. Chaque côté correspond à un type de terrain parmi *Road*, *City* ou *Field*. Le centre peut représenter une *City*, un *Field*, une *Banner* ou un *Monastery*.



FIGURE 2 – Exemple d'une tuile où des *Fields* possèdent un *Monastery*



FIGURE 3 – Exemple d'une tuile où une *City* possède une *Banner*

Pion

Lorsqu'un pion est placé sur une tuile, il prend un rôle en fonction du type de terrain occupé : *Knight* sur une *City*, *Farmer* sur un *Field*, *Thief* sur une *Road* et *Monk* sur un *Monastery*.

1.3.2 Déroulement d'un tour

Les joueurs jouent à tour de rôle selon un ordre prédéfini (généralement dans le sens horaire, en commençant par le joueur le plus jeune). Lors de son tour, un joueur effectue les actions suivantes dans l'ordre indiqué :

1. **Piocher une tuile :** Le joueur pioche une tuile dans la pioche actuellement active. Cette tuile est visible par l'ensemble des joueurs.
2. **Placer la tuile :** La tuile piochée doit être placée sur le plateau en respectant les règles suivantes :
   - La tuile doit être adjacente à au moins une autre tuile déjà posée.
   - Les côtés en contact doivent correspondre à des terrains de même type.
3. **Placer un pion (optionnel) :** Le joueur peut placer un pion sur un élément de terrain de la tuile qu'il vient de poser (*City*, *Road*, *Field*, *Monastery*).
4. **Restriction de placement des pions :** Si l'élément de terrain choisi est relié à un élément du même type déjà occupé par un pion, aucun nouveau pion ne peut y être placé. Les *Road* et *City* agissent comme des séparateurs entre les *Field*.
5. **Résolution des terrains complétés :** Si la pose de la tuile permet de compléter un ou plusieurs terrains, ceux-ci sont résolus immédiatement et les joueurs concernés marquent des points selon certaines règles. (voir partie 1.3.3)
6. **Retour des pions :** Tous les pions associés à des *City*, *Road* ou *Monastery* complétés durant le tour actuel sont retirés du plateau et rendus à leurs propriétaires. Les pions *Farmer* restent en jeu jusqu'à la fin de la partie.

1.3.3 Score

Les points sont attribués à deux moments :

- Pendant la partie : dès qu'un terrain (*City*, *Road*, *Monastery*) est complété (figure 4).
- À la fin de la partie : pour les terrains incomplets (*City*, *Road*, *Monastery*) ainsi que pour les *Field*.

Les joueurs ajoutent alors à leur score les points correspondant aux terrains selon les règles suivantes :

Road :

Une *Road* rapporte 1 point par tuile la composant au joueur possédant le plus de pions (*Thief*) sur cette *Road*. En cas d'égalité, tous les joueurs concernés marquent ces points.

City :

Une *City* complète rapporte 2 points par tuile qui la compose ainsi que 2 points par *Banner* présent, pour le joueur possédant le plus de pions (*Knight*). Si la *City* est incomplète à la fin de la partie, chaque tuile rapporte 1 point et chaque *Banner* rapporte également 1 point. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

Monastery :

Un *Monastery* rapporte 1 point par tuile ayant un sommet ou côté en commun, y compris la tuile du *Monastery*. Le score maximal d'un *Monastery* est donc de 9 points.

Field :

Chaque ensemble de *Fields* connectés ou isolés est attribué au joueur possédant le plus de pions (*Farmer*). Il marque alors 3 points par *City* complète adjacente aux *Field*. En cas d'égalité,tous les joueurs concernés marquent ces points.

A l'issue du décompte final, le joueur ayant obtenu le plus grand nombre de points remporte la partie.

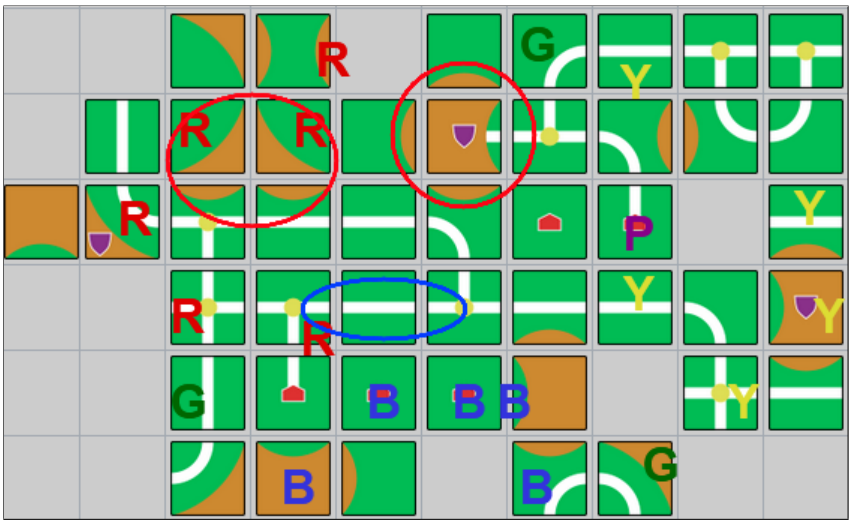


FIGURE 4 – Exemple de terrains complétés, avec deux *City* entourées en rouge et une *Road* entourée en bleu

1.4 Complexité

Bien que les règles de **Carcassonne** soient relativement simples, le jeu présente une richesse stratégique importante qui augmente à mesure que le plateau s'étend. Les choix offerts aux joueurs se multiplient progressivement en fonction de la configuration du plateau, des tuiles restantes et du placement des pions.

Contrairement à des jeux déterministes comme les échecs ou le shogi, **Carcassonne** introduit une composante stochastique via la pioche des tuiles, ce qui complexifie l'anticipation des coups futurs. Néanmoins, il demeure un jeu à information parfaite dans la mesure où l'ensemble des éléments placés sur le plateau est visible par tous les joueurs à tout instant.

La complexité combinatoire du jeu est particulièrement élevée pour plusieurs raisons. D'abord, chaque tuile peut être orientée de quatre manières, et chaque position libre sur le plateau peut être un emplacement légitime lorsque les côtés adjacents correspondent. Ensuite, la taille effective de l'espace d'états dépend de l'ordre de tirage des 72 tuiles, du placement des meeples et de la résolution des structures incomplètes. Sur un plateau en croissance, le nombre de successeurs possibles à un instant donné peut rapidement dépasser plusieurs dizaines, ce qui conduit à un arbre de jeu de taille astronomique. La taille de l'arbre de jeu est estimée à environ 10^{194}[6], ce qui rend toute exploration exhaustive irréaliste.

Mathématiquement, ce type de complexité se situe entre la simple permutation des tuiles et un vaste graphe de configurations : il faut combiner 72 choix de tuiles avec 4 orientations possibles et un grand nombre de positions valides, d'où un nombre de branches extrêmement élevé. D'où la nécessité d'un compromis entre profondeur de recherche, précision des heuristiques et temps de calcul.

Dans ce contexte, l'utilisation d'algorithmes de recherche heuristique et d'approximation apparaît comme une solution nécessaire pour proposer des décisions pertinentes dans un temps de calcul raisonnable.

2 Algorithmes et structures de données

Dans le cadre de ce projet, **Carcassonne** est considéré sous une forme restreinte pour la partie de réalisation d'intelligence artificielle afin de rendre possible l'utilisation d'algorithmes classiques de recherche dans les jeux.

Bien que **Carcassonne** ne soit pas strictement un jeu à somme nulle et qu'il comporte une composante stochastique liée à la pioche des tuiles, certaines hypothèses simplificatrices sont adoptées. En particulier, les décisions sont évaluées à partir de fonctions heuristiques estimant l'avantage relatif d'un joueur par rapport à son adversaire à un instant donné.

Cette modélisation permet l'utilisation d'algorithmes de recherche tels que Minimax et ses variantes, tout en restant consciente des limites théoriques de leur application à un jeu de cette nature.

Plusieurs algorithmes jouent ainsi un rôle central dans la réalisation des besoins du projet, notamment :

- **Minimax**[5], pour déterminer les coups optimaux en modélisant l'opposition entre les joueurs ;
- **Alpha-Beta**[7], qui permet de réduire le nombre d'états explorés en élaguant les branches non pertinentes de l'arbre de jeu ;
- **Iterative Deepening**[8], utilisé afin de borner le temps de calcul tout en améliorant progressivement la qualité des décisions ;
- **des fonctions d'évaluation heuristiques**, servant à estimer la qualité d'un état de jeu ;
- des approches basées sur *Monte Carlo Tree Search*[2], envisagées pour des stratégies alternatives ;
- des algorithmes de génération aléatoire, tels que le mélange de *Fisher–Yates*[4], nécessaires à la gestion équitable de la pioche.

La mise en œuvre efficace de ces algorithmes repose sur des structures de données adaptées permettant de représenter l'état du jeu et d'en faciliter la manipulation. Le plateau de jeu, constitué de tuiles connectées par leurs côtés, est modélisé en interne par un graphe de positions, offrant une représentation naturelle des relations d'adjacence entre les tuiles. Les états de jeu sont stockés sous forme d'objets si bien que chaque tuile possède un identifiant, un jeu de côtés, une rotation et des informations sur les meeples attachés.

La gestion de la pioche et du retour des pions utilise des structures séquentielles simples, comme des listes et des piles. Pour les calculs de score et la recherche de composantes connexes, des parcours en largeur (BFS) sur le graphe de tuiles sont employés, avec des ensembles de sommets visités pour éviter les redondances.

2.1 Structures de données

La représentation des données du jeu est un point clé du projet. En Python, le plateau est principalement géré par un dictionnaire de positions vers les tuiles, complété par une table d'adjacence pour les voisinages.

```
class Tile:
    code: str
```

```
    # Direction matrices indexed by orientation (0, 1, 2, 3)
    direction_matrix = [
        # 0 rotation
        [["NW", "N", "NE"], ["W", "C", "E"], ["SW", "S", "SE"]],
        # 1 rotation (90° clockwise)
        [["SW", "W", "NW"], ["S", "C", "N"], ["SE", "E", "NE"]],
        # 2 rotations (180° clockwise)
        [["SE", "S", "SW"], ["E", "C", "W"], ["NE", "N", "NW"]],
        # 3 rotations (270° clockwise)
        [["NE", "E", "SE"], ["N", "C", "S"], ["NW", "W", "SW"]],
    ]

    def __init__(
        self,
        code: str,
        connections: dict | None = None,
        connect_to: dict | None = None,
    ) -> None:
        self.code = code
        # tiles_lands is the terrain definitions for this tile
        self.tiles_lands = connections if connections is not None else {}
        self.connect_to = connect_to if connect_to is not None else {}
```

Cette structure permet de calculer rapidement les composantes connexes nécessaires au calcul des champs, des routes et des cités.

La pioche est mélangée avec l'algorithme *Fisher–Yates*[4] afin d'obtenir une permutation uniforme :

```
import random

  @staticmethod
  def _shuffle_fisher_yates_in_place(
      items: list[str], rng: random.Random
  ) -> None:
  for i in range(len(items) - 1, 0, -1):
    j = rng.randrange(i + 1)
    items[i], items[j] = items[j], items[i]

  @staticmethod
  def _shuffle_naive_in_place(items: list[str], rng: random.Random) -> None:
    n = len(items)
    for i in range(n):
      j = rng.randrange(n)
      items[i], items[j] = items[j], items[i]
```

Les états de partie sont ensuite définis par une classe 'GameState' contenant le plateau, la pioche restante, les scores et le nombre de meeples disponibles par joueur.

Dans la suite de cette section, nous nous concentrerons plus particulièrement sur les quatre premiers algorithmes présentés qui constituent le cœur du système de prise de décision implémenté dans le projet.

2.2 Algorithme Minimax

L'algorithme Minimax a pour but de déterminer les coups les plus optimaux pour un joueur à un état donné du jeu face à l'adversaire qui minimise le score du joueur "max". Ainsi, l'algorithme parcourt récursivement une partie de l'arbre des états du jeu, limitée par une profondeur maximale fixée afin de contenir la complexité de recherche. Chaque score d'une feuille est alors calculé via une fonction d'évaluation prédéfinie. Ce score remonte à son nœud parent qui sélectionne la valeur maximale ou minimale de ses nœuds enfants selon le joueur qu'il représente, et ainsi de suite jusqu'à la racine. L'algorithme possède 3 paramètres représentant l'état du jeu courant, la profondeur restante à explorer par rapport à l'état actuel, et un booléen à vrai lors du tour du joueur "max" et à faux sinon.

Voici le pseudocode de l'algorithme Minimax :

```
fonction minimax(etat, profondeur, joueur_max):
    SI profondeur = 0 OU estFeuille(etat) ALORS
        RETOURNER evalueHeuristique(joueur_max, etat)
    FIN SI
    SI joueur_max ALORS
        valeur = -infini
        POUR TOUT successeur s de etat FAIRE
            valeur = max(valeur, minimax(s, profondeur - 1, FAUX))
        FIN POUR TOUT
    FIN SI
    SINON
        valeur = +infini
        POUR TOUT successeur s de etat FAIRE
            valeur = min(valeur, minimax(s, profondeur - 1, VRAI))
        FIN POUR TOUT
    FIN SINON
    RETOURNER valeur
FIN FONCTION
```

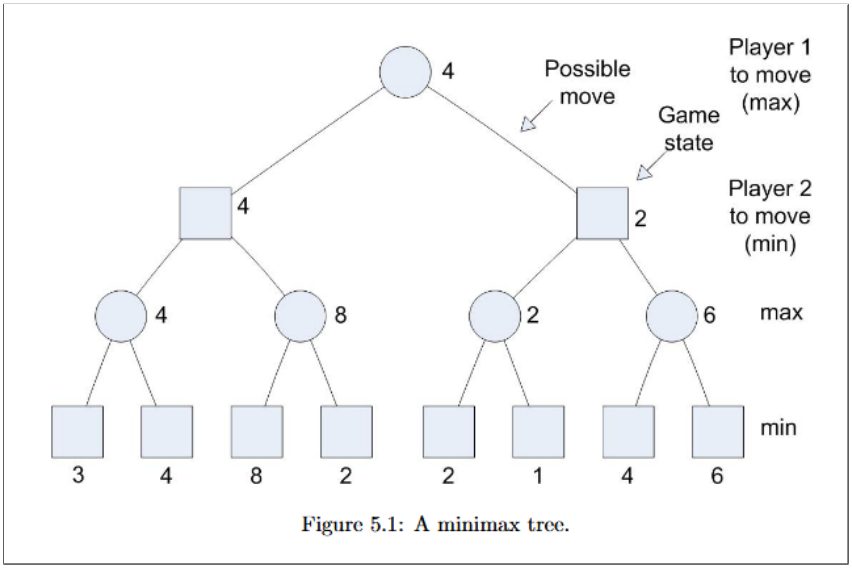


FIGURE 5 – Exemple d'un arbre de jeu partiel exploré par l'algorithme Minimax[6]

À la profondeur maximale à explorer, les feuilles remontent la valeur 4 (et non la valeur 3) lors du tour du joueur "max" pour maximiser son gain. Ainsi, au tour suivant, le joueur "min" récupère cette valeur 4 (et non la valeur 8) afin de minimiser le gain de son adversaire. Enfin, le joueur "max" récupère la valeur 4 (et non la valeur 2) pour maximiser son gain minimal. Il finit alors la recherche à la racine de l'arbre, ayant désormais connaissance d'une succession de coups optimaux à jouer pour obtenir un score de 4 en sa faveur à cette profondeur demandée.

2.3 Technique Iterative Deepening

L'algorithme Minimax avec approfondissement itératif (Iterative Deepening) est une technique de parcours progressif des profondeurs sur l'arbre des états d'un jeu. Après un parcours avec une nouvelle profondeur explorée, le meilleur coup pour le joueur "max" est enregistré en remplaçant le précédent, et sera alors renvoyé après épuisement du temps donné. Cette technique a pour avantage de déterminer une solution utile même sur une courte durée, tandis que l'exploration classique ne garantit pas que la profondeur fixée soit suffisante pour obtenir un résultat fiable après un temps bloquant.

En pratique, la recherche commence à faible profondeur puis augmente progressivement jusqu'à atteindre le temps limite :

```
fonction choix_coup(etat, temps_max):
    profondeur = 1
    meilleur_coup = null
    debut = temps()
    TANT_QUE temps() - debut < temps_max FAIRE
        meilleur_coup = recherche_minimax(etat, profondeur)
        profondeur += 1
    FIN TANT_QUE
    RETOURNER meilleur_coup
FIN FONCTION
```

Cette approche garantit qu'un coup plausible est toujours disponible à la fin du délai, même si la profondeur maximale n'a pas pu être atteinte.

2.4 Algorithme Alpha-Beta

Alpha-Beta (ou $\alpha\beta$-pruning) est une variante de l'algorithme Minimax consistant à explorer partiellement l'arbre des états d'un jeu en élaguant les branches qui n'apportent aucun gain nouveau pour le joueur "max".[7] En stockant la meilleure valeur courante pour le joueur maximal (α) et celle pour le joueur "min" (β) lors du parcours, l'algorithme n'évalue plus certains nœuds non-significatifs (et leurs potentiels enfants) quand $\alpha \geq \beta$, en réduisant potentiellement son temps d'exécution selon l'arbre donné.

Le principe est le suivant : dès qu'une branche devient trop mauvaise pour l'un des joueurs, elle ne peut plus influencer la décision finale et peut être abandonnée. Le gain en temps est particulièrement important lorsque les successeurs sont ordonnés de manière à évaluer d'abord les coups les plus prometteurs.

```
fonction alpha_beta(etat, profondeur, alpha, beta, joueur_max):
    SI profondeur = 0 OU estFeuille(etat) ALORS
        RETOURNER evalueHeuristique(etat)
    FIN SI
    SI joueur_max ALORS
        valeur = -infini
        POUR TOUT successeur s de etat FAIRE
            valeur = max(valeur, alpha_beta(s, profondeur - 1, alpha, beta, FAUX))
            alpha = max(alpha, valeur)
            SI alpha >= beta ALORS
                BRISER
            FIN SI
        FIN POUR TOUT
    SINON
        valeur = +infini
        POUR TOUT successeur s de etat FAIRE
            valeur = min(valeur, alpha_beta(s, profondeur - 1, alpha, beta, VRAI))
            beta = min(beta, valeur)
            SI beta <= alpha ALORS
                BRISER
            FIN SI
        FIN POUR TOUT
    FIN SINON
    RETOURNER valeur
FIN FONCTION
```

Cette version conserve la qualité de la décision de Minimax, tout en réduisant significativement le nombre de nœuds évalués lorsque le tri des coups est efficace.

2.5 Fonction d'évaluation heuristique

Étant donné l'impossibilité d'explorer l'arbre de jeu jusqu'à des états terminaux, l'algorithme Minimax repose sur une fonction d'évaluation heuristique permettant d'estimer la qualité d'un état de jeu intermédiaire. Cette fonction associe une valeur numérique à un état donné, représentant l'avantage relatif du joueur courant par rapport à son adversaire.

L'évaluation repose notamment sur les critères suivants :

- le score actuel des joueurs ;

- le potentiel de complétion des routes, cités et monastères en cours ;
- le nombre de pions disponibles pour chaque joueur ;
- la présence de pions bloqués sur des structures difficilement complétables ;
- le contrôle des champs adjacents à des cités complètes.

Ces différents éléments sont pondérés afin de refléter leur importance stratégique au cours de la partie. La fonction d'évaluation ne prétend pas fournir une estimation parfaite, mais permet de guider efficacement la recherche vers des coups jugés prometteurs.

2.6 Lien avec le projet

Les différentes techniques sont combinées dans le projet afin de rendre la recherche de coups pertinents possible malgré la complexité combinatoire élevée de Carcassonne. L'approche adoptée, mêlant Minimax, élagage Alpha-Beta, approfondissement itératif et fonctions d'évaluation heuristique, permet de guider efficacement la prise de décision des intelligences artificielles tout en respectant des contraintes de temps réalistes. La modélisation du plateau sous forme de graphes, associée à l'utilisation de structures de données adaptées comme les tables de hachage et les piles, assure une manipulation efficace de l'état du jeu et des configurations intermédiaires.

Ainsi, bien que l'exploration exhaustive de l'arbre de jeu reste impossible, le système mis en place fournit des conseils et des stratégies de qualité, exploitant de manière pragmatique les ressources computationnelles disponibles. Cette combinaison d'algorithmes et de structures de données constitue le cœur de l'intelligence artificielle du projet et offre une base solide pour l'extension future des fonctionnalités, telles que l'intégration de stratégies plus sophistiquées ou de variantes du jeu.

3 Architecture

Dans cette section, nous présentons l'architecture logicielle retenue pour l'implémentation du jeu Carcassonne. Le projet adopte une architecture n-tiers[1] visant à séparer clairement les responsabilités afin d'améliorer la maintenabilité, la testabilité et l'évolutivité du code. Le principe fondamental de cette architecture est de diviser le système en couches distinctes, chacune ayant des responsabilités spécifiques, et de limiter les dépendances au minimum grâce à des interfaces bien définies.

Dans un modèle n-tiers classique, chaque couche n'utilise que les services exposés par la couche située en dessous d'elle et ne dépend pas des détails d'implémentation de cette couche. Cela permet, par exemple, de remplacer l'interface utilisateur (CLI ou GUI) sans toucher à la logique métier, ou de simuler la persistance en tests unitaires en fournissant un faux service de stockage.

Ainsi, dans notre projet, nous avons choisi une architecture à trois tiers avec une séparation nette entre la présentation, la logique métier et les données. Cette architecture est pertinente dans notre cas car elle permet de faire évoluer le jeu sans mélanger les règles du jeu, l'affichage et la persistance des données. Elle est structurée autour de trois modules principaux : Présentation, Logique Métier et Données, organisés selon des dépendances unidirectionnelles.

3.1 Découpage en modules

Module Présentation

Le module Présentation est responsable des interactions avec l'utilisateur et de l'affichage de l'état du jeu. Il comprend notamment la gestion des commandes à travers le shell en CLI ou des actions à partir de la GUI, l'affichage du plateau, des tuiles, des meeples et des scores à travers CLI ou GUI.
Ce module ne contient aucune règle métier. Il délègue toute décision concernant la validité des actions ou l'évolution du jeu au module Logique Métier.

Responsabilités principales :

- Interprétation des actions utilisateur
- Appels aux interfaces de la logique métier
- Rafraîchissement de l'affichage

Module Logique Métier

Le module Logique Métier constitue le cœur du jeu. Il encapsule l'ensemble des règles du jeu et gère l'état interne de la partie. Il est notamment responsable de la mise à jour du plateau (via une liste d'adjacence), du calcul des scores, de la gestion des tours de jeu et des conditions de fin de partie. Il prend également en charge les actions des intelligences artificielles, à l'aide de fonctions d'évaluation, ainsi que la gestion de l'état du serveur et des parties en cours.
Ce module est totalement indépendant de l'interface graphique et de la persistance.

Responsabilités principales :

- Implémentation des règles du jeu
- Gestion de l'état de la partie
- Gestion des IA de jeu
- Gestion des actions liées au serveur

Module Données

Le module Données est dédié à la persistance et à la gestion des informations liées au jeu. Il s'occupe de la sauvegarde et du chargement des parties, de l'enregistrement de l'historique de jeu, ainsi que de la conservation des données relatives aux intelligences artificielles, notamment les structures de données et les meilleurs coups mémorisés dans le cadre de l'iterative deepening. Il gère également la persistance des parties en cours, de l'état du serveur et des informations associées aux joueurs.
Ce module n'implémente aucune règle métier et n'interagit jamais directement avec l'interface utilisateur.

Responsabilités principales :

- Sauvegarde des informations liées au jeu de près ou de loin (avec IA et serveurs) ;
- Possibilité de sauvegarder les parties en cours pour les recharger (dans des fichiers au format répondant aux fonctionnalités).

3.2 Dépendances entre modules

Les dépendances entre modules sont strictement unidirectionnelles (figure 6), conformément au principe d'architecture N-tiers.

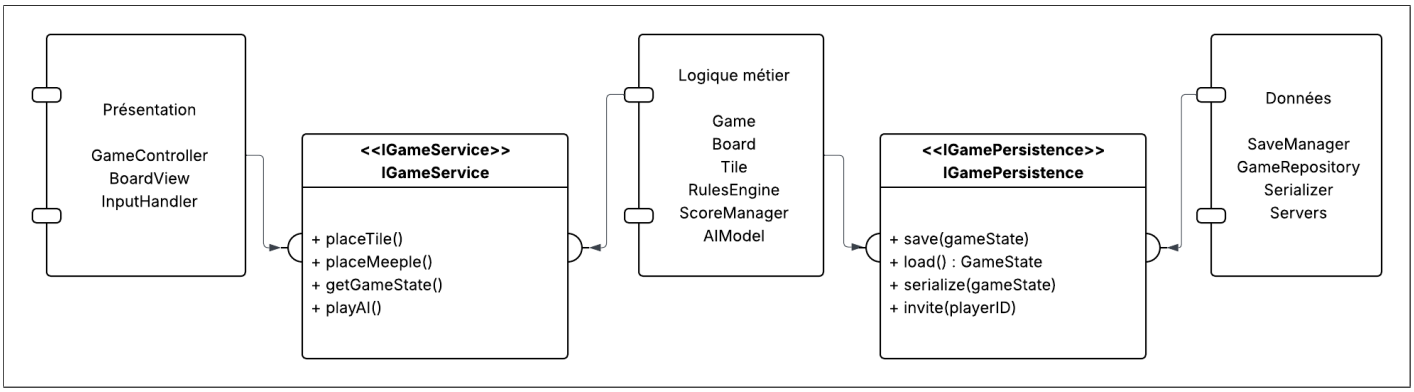


FIGURE 6 – Diagramme UML du projet

Ainsi, le module Présentation dépend uniquement du module Logique Métier, qu'il sollicite via des interfaces dédiées pour afficher l'état du jeu et transmettre les actions de l'utilisateur, sans accès direct aux données. Le module Logique Métier dépend uniquement du module Données pour la persistance des informations, cette interaction étant réalisée au moyen d'interfaces de persistance abstraites. Aucune dépendance directe n'existe entre la Présentation et les Données.

Cette organisation garantit une séparation claire des responsabilités et limite le couplage entre les composants.

3.3 Interfaces principales

Les interactions entre modules sont réalisées exclusivement via des interfaces bien définies.

Interface Présentation → Logique Métier

Exemples de services exposés :

```
place_tile(tile, position)
place_meeple(position)
get_game_state()
is_game_over()
```

Ces interfaces permettent à la présentation d'agir sur le jeu sans connaître l'implémentation interne des règles.

Interface Logique Métier → Données

Exemples de services exposés :

```
save_game(game_state)
load_game()
```

La logique métier délègue la persistance au module Données via ces interfaces abstraites.

4 Explication approfondie

Notre projet de Carcassonne en Python a été bien différent de ce que nous avions initialement prévu, notamment en raison de la complexité du jeu et des défis techniques rencontrés. Dans cette section, nous allons expliquer en détail les différentes étapes de notre développement, les choix que nous avons faits, ainsi que les difficultés que nous avons rencontrées pour deux fonctionnalités principales.

4.1 IA

4.2 Fichier de configuration

5 Revue critique du projet

Nous avons fait face à plusieurs défis au cours de ce projet notamment en ce qui concerne la complexité du jeu et la mise en œuvre de l'intelligence artificielle. Dans un premier temps, nous allons discuter de la completion du cahier des charges du projet. Dans un second temps, nous allons discuter des performances de notre projet et des limites de notre implémentation. Cette section terminera par une discussion sur les améliorations possibles pour notre projet.

5.1 Cahier des charges

Dans cette section, nous allons discuter de la manière dont nous avons respecté le cahier des charges du projet en analysant chaque partie et en expliquant ce qui a été fait et ce qui n'a pas été fait.

5.1.1 Généralités

Cette première partie du projet a été globalement respectée. Nous avons pris le temps avant de rendre le projet de vérifier le bon fonctionnement des tests et de vérifier que les différents bibliothèques utilisées étaient en accord avec les besoins. Des tests ont été mis en place tout au long du projet pour vérifier le bon fonctionnement de notre code et pour nous assurer que les différentes fonctionnalités étaient correctement implémentées. Le projet a alors une couverture globale de ?? ce qui est supérieur à ce qui était demandé dans le cahier des charges.

5.1.2 Besoins généraux

Cette partie a été relativement simple à respecter. La plus grande difficulté de cette partie a été de mettre en place le fichier de configuration qui est en lien avec tous les composants du projet auquel nous avons ajouté des fonctionnalités supplémentaires au fur et à mesure du projet.

5.1.3 Options au lancement

Cette partie a été simple à réaliser. Les besoins de cette partie n'était principalement que des options de lancements qui sont relativement simples à implémenter. Nous pouvons cependant parler des algorithmes de mélange de tuiles et de leur test statistique.

5.1.4 Gestion du jeu

La gestion des règles a été la grande première étape du projet mais également la première difficulté majeure. En effet, les règles de Carcassonne sont relativement complexes à implémenter. Entre la connectivité des tuiles,

les différentes conditions de placement des pions, les différentes conditions de scoring, il a fallu faire preuve de rigueur pour implémenter correctement les règles du jeu.

- Les règles de base ont été simples à implémenter.
- La gestion du score a été un grand défi. En effet, il a faullu mettre en place de la connexivité notamment pour les champs qui peuvent se situer entre des routes et des cités. Cette partie nous a pris beaucoup de temps et a été source de nombreux bugs, nous ayant alors ralentis dans le déroulement du projet.

5.1.5 Command Line Interface (CLI)

Le CLI a été relativement simple à implémenter, notamment grâce à l'utilisation de bibliothèques telles que *readline* de Python qui facilite la saisie de commande et de l'historique. En effet, l'importation de cette bibliothèque permet, sans grand effort, de valider deux besoins de cette partie :

- Édition de la commande en cours
- Historique des commandes tapées

Il a alors uniquement été nécessaire de faire des fonctions pour l'auto-complétion des commandes et de leurs arguments pour répondre au besoin F19 de complétion des commandes. Nous pouvons néanmoins penser à certaines améliorations possibles telles que la mise en point d'un design pattern pour la gestion des commandes du besoin F15. Ce design pattern aurait ainsi permis de faciliter la mise en place des commandes pour le GUI.

5.1.6 Format de sauvegarde

Cette partie s'est réalisé au fur et à mesure que nous avancions dans le projet. Nous avons d'abord implémenté une sauvegarde basique de l'état du jeu puis nous avons ajouté des fonctionnalités supplémentaires telles que le format de l'historique et la conformité avec le fichier de configuration. Cette partie aborde également le format du plateau de jeu qui a été relativement simple à implémenter, celui-ci n'étant pas très complexe à représenter de manière textuelle.

5.1.7 Structures de données

La mise en place d'un graphe pour représenter le plateau de jeu a été la première étapde du projet. Ce graphe nous a permis de gérer efficacement les différentes connexivités entre les tuiles notamment pour la gestion du score des champs en utilisant des composantes connexes. Toutefois, l'implémentation de ce graphe étant été faite au début du projet, il a été source de nombreux bugs et de difficultés dans la suite du projet. Nous pouvons notamment parler de la non existence de réelle classe *Tile* où ce qui est stocké dans le graphe est en réalité appelé *Vertex* ce qui a compliqué la gestion du code notamment lors de l'implémentation du GUI.

5.1.8 Graphical User Interface (GUI)

L'implémentation du GUI, avec l'intelligence artificielle, a été la partie la plus longue du projet à implémenter. En effet, le plus grand problème de cette partie a été d'attendre la mise en place des différentes fonctionnalités du projet pour pouvoir les implémenter sur le GUI. Cela a été plus difficile que le CLI car il a fallu mettre en place différents éléments graphiques tels que des menus, des boutons, des affichages de tuiles et de meeples. Une difficulté rencontrée par exemple a été de pouvoir poser une tuile sur le plateau de jeu. Le GUI utilise des coordonnées x et y pour représenter les positions sur le plateau de jeu alors que le module de logique métier utilise une tuile et une orientation pour représenter les positions sur le plateau de jeu. Il a alors fallu faire le lien entre ces deux représentations pour pouvoir poser une tuile sur le plateau de jeu à partir du GUI.

```
    def find_neighbor_at(
        self, x: int, y: int
```

```
    ) -> tuple[Optional[int], Optional[int]]:
        """
        Cherche une tuile voisine autour de (x, y).

        Retourne un tuple (vertex_id, side_relative_to_new_tile)
        side_relative_to_new_tile : côté de la nouvelle tuile où est collé
        le voisin
        """
        # Mapping direction → offset
        neighbors_offset = {
            NORTH_DIR: (0, -1),
            EAST_DIR: (1, 0),
            SOUTH_DIR: (0, 1),
            WEST_DIR: (-1, 0),
        }

        for side, (dx, dy) in neighbors_offset.items():
            nx, ny = x + dx, y + dy
            for vertex in self.vertices.values():
                vx, vy = vertex.get_position()
                if (vx, vy) == (nx, ny):
                    # side représente le côté de la nouvelle tuile où ce voisin
                    # est collé
                    opposite_side = (side + 2) % 4
                    return vertex.get_id(), opposite_side
```

Cette fonction permet de trouver une tuile voisine autour d'une position donnée (x,y) sur le plateau de jeu qui sera utilisée pour le *play_move* du module de logique métier.

Malgré tous ces défis, il reste quelques améliorations possibles pour le GUI pour le rendre cohérent avec tous les besoins du projet.

5.1.9 Joueur artificiel

La partie sur les joueurs artificiels a été la partie la plus dure du projet. En effet, comme abordé dans la section 2.2, Carcassonne est un jeu à la fois stochastique et à information parfaite ce qui rend l'implémentation de Minimax inutilisable dans sa forme classique. Il a fallu ainsi implémenter une version adaptée de Minimax avec des noeuds chance pour prendre en compte la pioche des tuiles. Ceci correspond alors à l'algorithme Expectimax, qui est une extension de Minimax pour les jeux stochastiques. Cependant, l'implémentation de cet algorithme s'est avérée très complexe et a été source de nombreux bugs, notamment en ce qui concerne la gestion des noeuds chance et la mise en place de fonctions d'évaluation heuristiques adaptées à ce jeu.

5.1.10 Jeu en réseau

Ce qui nous a pris le plus de temps dans cette partie a été de gérer le serveur côté GUI. Plus précisément, il a fallu faire en sorte que le GUI affiche les messages envoyé par le serveur. Nous avons alors du mettre en place le design pattern Visitor pour faire en sorte que le GUI puisse afficher les messages du serveur sans avoir à faire de lien direct entre les deux composants.

5.2 Performance

5.3 Limites et bugs

Toutefois, malgré nos efforts pour implémenter correctement les différentes fonctionnalités du projet, nous avons rencontré de nombreux bugs et limitations au cours du développement.

5.4 Perspectives d'amélioration

Une fois le projet complété, nous avons identifié de nombreuses pistes d'amélioration que ce soit au niveau de l'implémentation de certaines fonctionnalités, de la mise en place de notre code ou bien l'extensivité de notre projet.

L'amérioration la plus évidente serait la mise en forme de notre code. En effet, notre code est relativement mal organisé et peu lisible ce qui rend la maintenance et l'évolution du projet plus difficile. Il aurait été ainsi intéressant de mettre en place un design pattern pour la gestion des commandes par exemple, ou bien de structurer notre code de manière plus claire et lisible. Cela a été du à un manque d'imagination de notre part pour prévoir une implémentation claire et lisible pour le futur. Nous aurions du consacrer plus de temps à la réflexion sur l'implémentation de notre code avant de nous lancer dans le développement ce qui nous aurait permis d'avoir une meilleure vision d'ensemble du projet et ainsi éviter de faire des choix d'implémentation qui ont rendu notre code plus difficile à maintenir et à faire évoluer. Nous avons essayé de remédier à ce problème au fur et à mesure du projet mais nous avons parfois été contraints de faire des compromis pour avancer plus rapidement. Cela a été le cas de notre fichier *game.py* qui était au départ relié à beaucoup trop de fonctionnalités différentes, ou bien au fichier *gui.py* qui était lui aussi relié à beaucoup de fonctionnalités différentes concernant le GUI.

5.5 Utilisation de l'IA

Tout au long du projet, nous avons eu recours à l'utilisation de l'intelligence artificielle pour de nombreuses tâches. Que ce soit pour la mise en place d'un code de base pour une partie du projet, améliorer la qualité du code ou bien affiner certaines fonctionnalités, l'IA a été un outil précieux pour nous aider à avancer plus rapidement et efficacement. Nous avons ainsi appris à utiliser différents modèles d'IA pour différentes tâches en fonction de leurs forces et de leurs faiblesses.

Nous avons beaucoup utilisé l'extension *Copilot* de Visual Studio Code qui nous a permis d'obtenir des suggestions de code pertinentes et de gagner du temps dans les tâches de codage répétitives en raison de sa capacité à avoir accès au contexte du code. L'utilisation d'autres modèles d'IA tels que *ChatGPT* ou *Claude 4.6* a également été bénéfique pour la mise en place de certaines fonctionnalités plus complexes telles que l'affichage de la réponse du server dans le GUI.

6 Liste des besoins

Besoin	Description du besoin	Statut
F1	Lancement du programme	Non fait
F2	Fichier de configuration	Non fait
F3	Internationalisation (i18n)	Non fait
F4	Interface en ligne de commande	Non fait
F5	Interface graphique	Non fait

Besoin	Description du besoin	Statut
F6	Mode blitz	Non fait
F7	Mode contest	Non fait
F8	Graine aléatoire	Non fait
F9	Mélange des tuiles	Non fait
F10	Options spécifiques	Non fait
F11	Gestion des règles	Non fait
F12	Gestion des erreurs utilisateur	Non fait
F13	Gestion du undo/redo	Non fait
F14	Gestion des conseils	Non fait
F15	Shell interactif	Non fait
F16	Proposer de sauvegarder avant quitter	Non fait
F17	Édition de la commande en cours	Non fait
F18	Historique des commandes tapées	Non fait
F19	Complétion des commandes	Non fait
F20	Notation des coups	Non fait
F21	Sauvegarde et restauration d'une partie	Non fait
F22	Commentaires dans la sauvegarde	Non fait
F23	Format des paramètres du jeu	Non fait
F24	Format du plateau de jeu	Non fait
F25	Format de l'historique	Non fait
F26	Module *graph*	Non fait
F27	Algorithmes de manipulation des graphes	Non fait
F28	Menus de l'application	Non fait
F29	Raccourcis clavier	Non fait
F30	Drag'n'drop	Non fait
F31	Temps de réflexion borné	Non fait
F32	Minimax avec $\alpha\beta$-pruning	Non fait
F33	Fonctions d'évaluation pour Minimax	Non fait
F34	Approfondissement itératif de Minimax	Non fait
F35	Profondeur de recherche de Minimax	Non fait
F36	Monte Carlo Tree Search (MCTS)	Non fait
F37	Fonction de sélection par Machine Learning	Non fait
F38	Découverte de serveurs et connexion client- serveur basique	Non fait
F39	Serveur multi-clients et gestion de parties	Non fait
F40	Système d'invitations et gestion avancée des joueurs	Non fait

Références

[1] Nawar S Alseelawi, Enas K Adnan, Hussein T Hazim, Haider Alrikabi, and Khalid Nasser. Design and implementation of an e-learning platform using n-tier architecture. *International Journal of Interactive Mobile Technologies (iJIM)*, 2020.

[2] Fred Valdez Ameneyro, Edgar Galván, and Ángel Fernando Kuri Morales. Playing carcassonne with monte carlo tree search. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 2343–2350. IEEE, 2020.

[3] Christopher Obe. Carcassonne complete annotated rules. https://www.dover.nh.gov/Assets/government/city-operations/library/borrow/library-of-things/carcassonne-rules.pdf, 2014.

[4] Ronald A. Fisher and Frank Yates. Statistical tables for biological, agricultural and medical research. *Oliver and Boyd*, 1938.

[5] Thomas Hauk, Michael Buro, and Jonathan Schaeffer. Rediscovering*-minimax search. In *International Conference on Computers and Games*, pages 35–50. Springer, 2004.

[6] Cathleen Heyden. Implementing a computer player for carcassonne. *Master's thesis, Department of Knowledge Engineering, Maastricht University*, 2009.

[7] Donald E. Knuth and Ronald W. Moore. Analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4) :293–326, 1975.

[8] Richard E. Korf. Depth-first iterative-deepening : An optimal admissible tree search. *Artificial Intelligence*, 27(1) :97–109, 1985.